NetShield-ML

Two-System Demonstration Workflow and Computer Networks Concepts

Project: Distributed Network Intrusion and Attack Detection System

Purpose: Demonstrate communication between two physical systems, network traffic monitoring, attack simulation, and hybrid threat detection.

---

1. Project Objective

NetShield-ML is a distributed Network Intrusion Detection System (NIDS). It monitors network traffic, aggregates packets into flows, extracts security-related features, applies heuristic rules and a Random Forest model, and presents the results in a React dashboard.

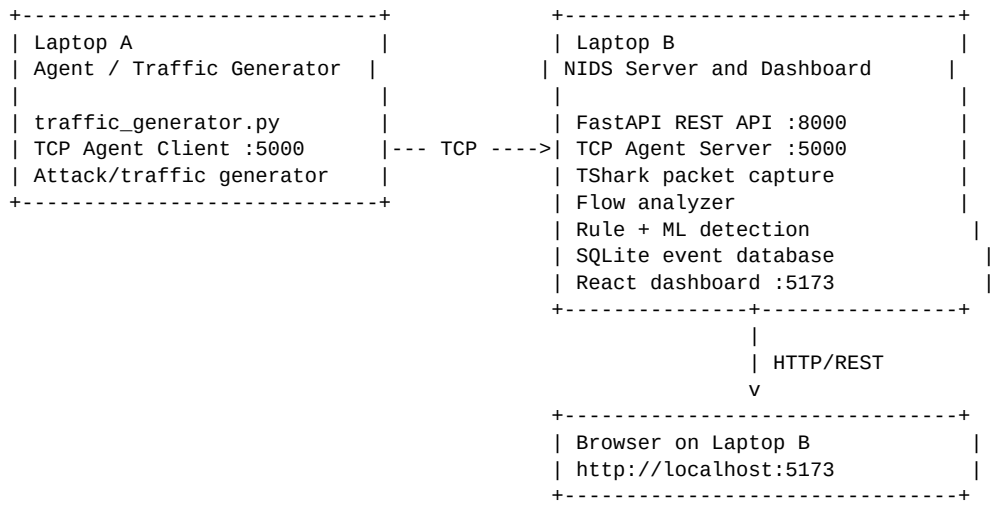
The demonstration uses two systems:

- **Laptop A: Agent / Traffic Generator**
- **Laptop B: NIDS Server / Monitoring Dashboard**

The project demonstrates both real network communication and controlled synthetic traffic injection for repeatable testing.

---

2. Two-System Architecture



Port allocation

Port	Service	Direction
`5000/TCP`	Custom NetShield agent protocol	Laptop A to Laptop B
`8000/TCP`	FastAPI REST API	Browser/frontend to Laptop B
`5173/TCP`	Vite React development server	Browser to Laptop B

The TCP agent channel and captured traffic are separate concepts. The agent channel proves system-to-system communication; TShark observes packets on the selected network interface for traffic analysis.

---

3. Responsibilities of Each System

Laptop A: Agent

Laptop A runs `agent/traffic_generator.py`. It can:

- Connect to Laptop B using TCP.
- Send a HELLO message and receive an acknowledgement.
- Send periodic heartbeats every three seconds.
- Measure communication latency.
- Return acknowledgements for server probes.
- Generate normal traffic.
- Simulate port scans, traffic spikes, and TCP connection bursts.

Laptop B: NIDS Server

Laptop B runs the main NetShield services. It:

- Listens for the agent on TCP port `5000`.
- Runs the FastAPI API on port `8000`.
- Captures packets with TShark when configured.
- Aggregates packets into five-tuple flows.
- Extracts rate, port, SYN, RST, and packet-size features.
- Applies rule-based and ML detection.
- Stores security events in SQLite.
- Serves telemetry to the React dashboard.

---

4. Preparation

4.1 Connect both systems

Connect Laptop A and Laptop B to the same Wi-Fi network, Ethernet switch, or hotspot. Both systems must be able to reach each other over the LAN.

4.2 Find Laptop B's LAN address

On Laptop B:

Windows

```
ipconfig
```

Find the IPv4 address of the active adapter, for example `192.168.1.50`.

Linux

```
ip -br addr
```

Refer to this address as `<LAPTOP_B_IP>`.

4.3 Install dependencies

On both systems:

```
python -m venv venv
```

Activate the environment.

Linux/macOS

```
source venv/bin/activate
```

Windows PowerShell

```
.\venv\Scripts\Activate.ps1
```

Install Python dependencies:

```
pip install -r requirements.txt
```

On Laptop B, install frontend dependencies:

```
npm install
```

4.4 Allow firewall ports on Laptop B

The server must accept inbound TCP connections on ports `5000` and `8000`.

Linux with UFW

```
sudo ufw allow 5000/tcp
sudo ufw allow 8000/tcp
```

Windows PowerShell as Administrator

```
New-NetFirewallRule -DisplayName "NetShield Agent" -Direction Inbound -LocalPort 5000 -Protocol TCP -Action A
```

```
New-NetFirewallRule -DisplayName "NetShield API" -Direction Inbound -LocalPort 8000 -Protocol TCP -Action All
```

From Laptop A, test the agent port:

Linux/macOS

```
nc -zv <LAPTOP_B_IP> 5000
```

Windows PowerShell

```
Test-NetConnection <LAPTOP_B_IP> -Port 5000
```

---

5. Starting the Demonstration

Step 1: Start the backend on Laptop B

From the project root on Laptop B:

```
python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000
```

This starts the FastAPI service, the TCP agent server, the detection engine, and the database layer.

Expected services:

```
FastAPI API:       0.0.0.0:8000
Agent TCP server:  0.0.0.0:5000
```

Step 2: Start the React dashboard on Laptop B

Open another terminal:

```
npm run dev
```

Open the displayed Vite URL, normally:

```
http://localhost:5173
```

The top status indicator should show that the backend is connected.

Step 3: Start the agent on Laptop A

```
python agent/traffic_generator.py --server <LAPTOP_B_IP> --port 5000
```

Example:

```
python agent/traffic_generator.py --server 192.168.1.50 --port 5000
```

The expected protocol sequence is:

```
Laptop A -> Laptop B: TCP connection request
Laptop A -> Laptop B: hello
Laptop B -> Laptop A: hello_ack
```

```
Laptop A -> Laptop B: heartbeat
Laptop B -> Laptop A: heartbeat_ack
```

Heartbeats continue approximately every three seconds.

Step 4: Verify the agent

On Laptop B, open the **Devices** page and confirm:

- Agent status is `CONNECTED`.
- Remote hostname and IP are visible.
- Heartbeat time updates.
- RTT latency is displayed.

This confirms the custom TCP control channel is working.

---

6. Configuring Live Packet Capture

Live capture is optional for the controlled testbed, but it is required to observe real packets from the network interface.

On Laptop B, check interfaces:

```
tshark -D
```

Set the interface in `.env`:

```
CAPTURE_INTERFACE=eth0
```

A Windows configuration might be:

```
CAPTURE_INTERFACE=Wi-Fi
```

The Monitoring page can also expose available interfaces when TShark is installed.

On Linux, capture permissions may require membership in the `wireshark` group. On Windows, Npcap must be installed and running.

If TShark is not available, use the dashboard simulation controls. Synthetic packets are sent through the same flow analyzer, feature extractor, and detection engine.

---

7. Demonstration Workflow

Phase 1: Normal communication

1. Start the backend and dashboard on Laptop B.
2. Start the agent on Laptop A.

3. Show the Devices page and explain the HELLO/ACK exchange.

4. Wait for heartbeat messages.

5. Point out the measured RTT.

Phase 2: Normal traffic

From the agent menu, select the normal mixed traffic option, or use the dashboard simulation panel.

On the Monitoring page, observe:

- Total packets
- Packets per second
- Bandwidth
- TCP, UDP, and ICMP distribution
- Active network flows
- Five-tuple flow entries

Phase 3: Port scan detection

Run the port scan scenario from Laptop A or the Security page.

The demonstration sends SYN traffic to many destination ports. Expected results include:

- More than 15 unique destination ports.
- `PORT_SCAN_LIKE` heuristic alert.
- Possible `ML_PORT_SCAN` classification.
- Source IP and destination port information in the event details.

Phase 4: Traffic spike detection

Run the traffic spike scenario.

The demonstration sends a high-volume burst of packets. Expected results include:

- Increased packet rate.
- Increased bandwidth.
- `TRAFFIC_SPIKE` rule alert.
- Possible `ML_TRAFFIC_SPIKE` classification.

Phase 5: SYN connection burst

Run the connection burst scenario.

The demonstration generates many TCP SYN packets in a short period. Expected results include:

- Increased connection rate.
- High SYN ratio.

- `CONNECTION_BURST` rule alert.
- Possible `ML_SYN_BURST` classification.

Phase 6: Investigate and export

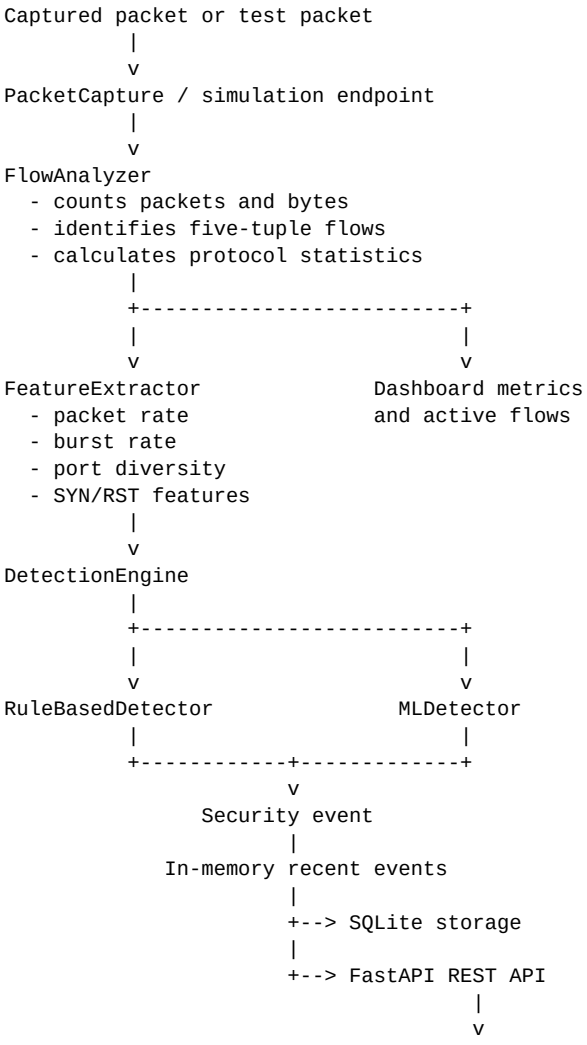
On the Security page:

1. Open an alert in the event queue.
2. Inspect source and destination endpoints.
3. Review protocol and severity.
4. Review the detection reason.
5. Review ML confidence and feature values when available.
6. Inspect the suggested firewall mitigation command.
7. Export the event list as CSV.

---

8. Internal Processing Workflow

The complete backend workflow is:



```
React dashboard
```

---

9. Computer Networks Concepts Demonstrated

Client-server architecture

Laptop B listens as the server. Laptop A initiates the connection as the client.

IPv4 addressing and LANs

Private IPv4 addresses identify systems inside the local network. The machines must have network reachability between them.

TCP

TCP is used for the agent channel and provides:

- Connection establishment
- Reliable delivery
- Ordered bytes
- Retransmission
- Full-duplex communication
- Connection termination

TCP three-way handshake

Before the NetShield application messages are exchanged, TCP performs:

```
SYN ->
     <- SYN-ACK
ACK ->
```

Port numbers and multiplexing

Ports allow multiple services to run on the same Laptop B without confusing their traffic. NetShield separates the agent protocol, REST API, and dashboard server.

UDP

The traffic spike simulator uses UDP because it is connectionless and has low protocol overhead. It is suitable for demonstrating rapid packet transmission.

Application-layer protocol

NetShield defines its own newline-delimited JSON protocol containing message types such as `hello`, `heartbeat`, and `probe`.

TCP is a byte stream, so the newline delimiter provides application-level message framing.

Heartbeats and failure detection

Periodic heartbeats allow the server to determine whether the remote agent is still reachable and active.

Round-trip time

The time between sending a message and receiving its acknowledgement is used to estimate network latency.

Packet capture and passive monitoring

TShark observes frames from a network interface and extracts fields from IP, TCP, and UDP headers.

Five-tuple flow identification

A network flow is identified by:

```
Source IP, destination IP, source port, destination port, protocol
```

TCP flags

The project uses SYN, ACK, and RST flags to identify connection attempts, successful communication patterns, and failed or reset connections.

Throughput and rate measurement

The system calculates packets per second, bytes per second, and bandwidth in Mbps.

Sliding-window analysis

Recent traffic is evaluated over 30-second and 2-second windows. This supports detection of both sustained activity and short bursts.

HTTP and REST APIs

The React application obtains status, metrics, flows, events, and model information through HTTP GET and POST requests to FastAPI.

CORS and separated frontend/backend services

The frontend and backend run on different ports, so Cross-Origin Resource Sharing is enabled in FastAPI.

Concurrency

Background threads handle packet capture, TCP client communication, heartbeat processing, detection, and chart recording concurrently.

Firewall and access control

The demonstration also illustrates how host firewalls control inbound access to TCP ports and can prevent otherwise-correct network applications from communicating.

---

10. Short Presentation Script

> NetShield uses two systems. Laptop A acts as the traffic-generating agent, and Laptop B acts as the monitoring server. Laptop A connects to Laptop B over TCP port 5000, exchanges HELLO and heartbeat messages, and provides RTT measurements. Laptop B captures network traffic, aggregates packets into five-tuple flows, extracts features such as packet rate, port diversity, SYN ratio, and RST count, and evaluates those features using both heuristic rules and a Random Forest model. The FastAPI service exposes the results, and the React dashboard displays device status, traffic metrics, active flows, and security alerts. We first demonstrate normal communication, then generate a port scan, traffic spike, and SYN burst to show how the monitoring and detection pipeline responds.

---

11. Troubleshooting Checklist

Symptom	Likely cause	Check
Connection refused on port 5000	Backend not running or firewall block	Start FastAPI and allow TCP 5000
Timeout from Laptop A	Wrong IP or different network	Verify `<LAPTOP_B_IP>` and LAN connectivity
Dashboard says disconnected	API unavailable	Open `http://localhost:8000/api/status`
No live packets	TShark/interface/permissions issue	Run `tshark -D` and configure `CAPTURE_INTERFACE`
Agent connected but no captured attack	Capture is disabled or wrong interface	Enable TShark capture on Laptop B
Port 8000 unavailable	Another service is using the port	Stop the service or choose another API port
No alerts from a test	Detection cooldown or threshold not reached	Clear events and repeat a controlled simulation

---

12. Important Scope Note

NetShield is a functional educational NIDS prototype. Its rules are heuristic, and its Random Forest model is trained on synthetic traffic patterns. The project demonstrates the architecture and workflow of network monitoring and intrusion detection; it should not be presented as a production firewall or a complete real-world threat detection solution.

---

13. Relevant Source Files

- `backend/main.py` - FastAPI application and pipeline orchestration
- `backend/networking/server.py` - TCP server and heartbeat protocol
- `backend/networking/client.py` - TCP agent client
- `agent/traffic_generator.py` - agent and traffic simulations
- `backend/monitoring/packet_capture.py` - TShark integration
- `backend/monitoring/flow_analyzer.py` - five-tuple flow aggregation
- `backend/detection/features.py` - feature extraction
- `backend/detection/rules.py` - heuristic detection rules
- `backend/detection/ml_detector.py` - Random Forest detector
- `backend/database/database.py` - SQLite persistence
- `src/api.js` - frontend-to-backend API client
- `src/pages/MonitoringPage.jsx` - monitoring interface
- `src/pages/SecurityPage.jsx` - alerts and simulation interface
- `src/pages/DevicesPage.jsx` - two-system connectivity interface